CS F429 NLP — Phase 2 Report

TEAM: 1
Guru Sai Prasad Mallepalli Gajula: 2023A7PS0156H
Haneesh Sai Kankipati: 2023A7PS0052H
Divyajot Singh: 2023A7PS0005H
Utkarsh Singh: 2023AAPS0253H

Code: two implementations of the classical baseline — Run 1 (`data.py`, `classical.py`, `analyze.py`) and Run 2 (`bt_baseline.py`, `features.py`, `data.py`). Section 7 gives the run procedure.

1. Introduction

Task: pairwise preference prediction. Given a prompt x and two candidate responses y_A, y_B, predict which one a human (or AI) annotator prefers. Every model in this project produces a scalar margin $m = r(x, y_A) - r(x, y_B)$ and predicts A iff $m > 0$. Under the Bradley-Terry model the margin is a log-odds:

$$P(y_A \succ y_B \mid x) = \sigma(m)$$

This is the decision at the core of RLHF reward models, DPO, best-of-n sampling and LLM-as-a-judge pipelines. The project question (Phase 1) is whether a frozen model's hidden representations carry preference signal beyond surface heuristics and fine-tuned language-model rewards. Phase 2 builds the first rung of that comparison: a **classical Bradley-Terry baseline over surface features**, with a length-only control, evaluated on held-out pairs with error analysis.

2. Literature Survey

Pairwise preference models. Bradley & Terry (1952) model the probability that item i beats item j as $\sigma(r_i - r_j)$. Every reward model in modern RLHF is a Bradley-Terry model whose utility r is a neural network; our classical baseline uses the same likelihood with a linear utility over hand-crafted features.

RLHF. Christiano et al. (2017) learned reward models from human pairwise comparisons and optimized policies against them with RL. Stiennon et al. (2020) applied this to summarization, and Ouyang et al. (2022, InstructGPT) scaled it to instruction following with PPO (Schulman et al., 2017). Bai et al. (2022) released the Anthropic HH-RLHF helpfulness/harmlessness comparisons used here for transfer.

Direct Preference Optimization. Rafailov et al. (2023) showed the RLHF objective has a closed-form optimal policy, so the policy itself defines an implicit reward $\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ and can be trained directly on preference pairs without a separate reward model or RL loop. DPO with parameter-efficient LoRA adapters (Hu et al., 2022) is now the standard low-cost alignment recipe (e.g. Zephyr, Tunstall et al., 2023, trained on binarized UltraFeedback).

Why the problem matters: length and style confounds. Preference data and the models trained on it reward verbosity. Singhal et al. (2023) found much of RLHF's reward improvement is explained by output length; Park et al. (2024) showed DPO exploits length and proposed a length-regularized variant; Dubois et al. (2024) introduced length-controlled AlpacaEval because LLM judges favour longer answers, a verbosity bias also reported by Zheng et al. (2023). RewardBench (Lambert et al., 2024) benchmarks

reward models on exactly our task. These works motivate reporting every model against a **length-only control**, which we do throughout.

References

- Bai, Y. et al. (2022). Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862.
- Bradley, R. A. & Terry, M. E. (1952). Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika* 39(3/4).
- Christiano, P. et al. (2017). Deep Reinforcement Learning from Human Preferences. *NeurIPS*.
- Cui, G. et al. (2023). UltraFeedback: Boosting Language Models with High-quality Feedback. arXiv:2310.01377.
- Dubois, Y. et al. (2024). Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. arXiv:2404.04475.
- Hu, E. et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.
- Lambert, N. et al. (2024). RewardBench: Evaluating Reward Models for Language Modeling. arXiv:2403.13787.
- Ouyang, L. et al. (2022). Training Language Models to Follow Instructions with Human Feedback. *NeurIPS*.
- Park, R. et al. (2024). Disentangling Length from Quality in Direct Preference Optimization. *Findings of ACL*.
- Rafailov, R. et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *NeurIPS*.
- Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Singhal, P. et al. (2023). A Long Way to Go: Investigating Length Correlations in RLHF. arXiv:2310.03716.
- Stiennon, N. et al. (2020). Learning to Summarize from Human Feedback. *NeurIPS*.
- Tunstall, L. et al. (2023). Zephyr: Direct Distillation of LM Alignment. arXiv:2310.16944.
- Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets & Benchmarks*.

3. Dataset and Preprocessing

3.1 Datasets

	UltraFeedback (binarized)	**HH-RLHF**
Source	`trl-lib/ultrafeedback_binarized` (HF Hub), from Cui et al. (2023)	`Anthropic/hh-rlhf` (HF Hub), Bai et al. (2022)
Domain	General instruction following, reasoning, coding, chat	Multi-turn assistant dialogue: helpfulness and harmlessness
Size	62,135 train / 1,000 test pairs	8,552 test pairs (4 subsets)
Labels	GPT-4 ratings (1–10); chosen = higher-rated of two completions	Human pairwise choice
Role	Training + in-domain evaluation	Cross-corpus transfer evaluation only

Each UltraFeedback row is `{chosen, rejected, score_chosen, score_rejected}`, where `chosen`/`rejected` are chat-message lists ending in the assistant response.

3.2 Initial observations (`python data.py`)

Statistic	Train	Test
Pairs	62,135	1,000
Tied scores (`score_chosen == score_rejected`)	7,554 (12.2%)	105 (10.5%)
Median words, chosen / rejected	158 / 130	167 / 130.5
Chosen response is longer	53.5%	55.3%
Score gap ≤ 0.5 (near-ties)	30.6%	29.6%
Multi-turn conversations	0	0

Observations that shaped the design:

- **Length skew:** chosen responses are longer in a majority of pairs, so a "pick the longer one" rule already beats chance. Every result is reported next to a length-only control.
- **Ties:** 12% of pairs carry no preference. They are dropped from training and reported as a separate slice at test time.
- **Label noise:** ~30% of pairs differ by ≤ 0.5 points on a 10-point scale, which caps attainable accuracy well below 100%.
- **Prompt-disjointness:** we checked that 0 of the 1,000 official test prompts appear in the training split, so the official split has no prompt leakage.

3.3 Preprocessing

Surface features are computed on raw text; no stemming or stopword removal (they would distort the length and formatting signals being measured).

1. **Flattening.** Prompt = all non-assistant turns joined; response = final assistant turn.
2. **Tie handling.** Drop tied pairs from training; keep them in test with `is_tie = 1`.
3. **A/B randomization.** Test pairs are shown as (A, B) with order randomized by a fixed seed, so `label` (1 iff A is chosen) is not constant and a position bias cannot score.
4. **Tokenization.** Word tokens via regex `\w+` (lowercased) for overlap features; whitespace split for word counts; TF-IDF tokenization from scikit-learn.
5. **Cleaning (Run 2 additionally):** drop empty, identical-response and exact-duplicate pairs; log-length uses the `Qwen/Qwen2.5-1.5B-Instruct` tokenizer so it matches the language model used in later phases.

Example (official test split, shortest non-tie pair; prompt truncated):

```
raw chosen:   [{role: user, content: "In this task, you're given two sentences. Indicate
if
               the first sentence clearly entails the second ... Answer:"},
               {role: assistant, content: "0"}]           score_chosen   = 8.0
raw rejected: [... same user turn ...,
               {role: assistant, content: "1"}]           score_rejected = 6.0

after preprocessing:
  prompt = "In this task, you're given two sentences. ... Answer:"
  A = "1", B = "0"  (order randomized)   label = 0   is_tie = 0
  len_chosen = 1, len_rejected = 1  -> length features give margin 0; only content can
decide
```

4. Method

4.1 Classical baseline: Bradley-Terry logistic regression

$$P(y_A \succ y_B \mid x) = \sigma\big(w^\top [f(x, y_A) - f(x, y_B)]\big)$$

- **Features** $f(x, y)$ (per prompt–response):

Feature	Definition
`n_words`	response word count
`n_lines`	response line count
`has_code`	1 if response contains a ``` code block
`prompt_coverage`	fraction of prompt word types that appear in the response
`tfidf_cos`	cosine of TF-IDF vectors of prompt and response

- **Symmetry.** Train on mirrored pairs $[d, -d]$ with labels $[1, 0]$ where $d = f(\text{chosen}) - f(\text{rejected})$, and use **no intercept**. This gives $m(A, B) = -m(B, A)$ exactly, so the model cannot learn a position bias.
- **Training.** Features standardized (`StandardScaler`, fit on train; the mirrored data has zero mean). `LogisticRegression(fit_intercept=False, C=1.0, lbfgs, max_iter=1000)`. TF-IDF: `sublinear_tf=True, min_df=2`, fit on training prompts and responses only.
- **Length-only control.** Same model with `n_words` as the only feature.
- **Why this baseline.** It is the pre-transformer version of exactly what a neural reward model does (same Bradley-Terry likelihood, linear utility), it is interpretable through its weights, and it directly measures how much of the task surface statistics already solve.
- **Implementation.** Python 3.12, scikit-learn, Hugging Face `datasets`; CPU only (MacBook), trains on 54,581 pairs in about a minute.

Second implementation (Run 2, `bt_baseline.py`). Same model family with a different feature set — `log_len` (Qwen tokens), `n_lines`, `n_bullets`, word Jaccard, prompt-overlap fraction, TF-IDF cosine — a

different split (HF train and test pooled, then an 80/10/10 prompt-disjoint split by MD5 hash of the prompt; 43,973 train / 4,000 val / 4,000 test pairs), C tuned on validation over {0.01, 0.1, 1, 10}, and an additional HH-RLHF transfer test. Two implementations with different features and splits agreeing is itself a robustness check.

4.2 Evaluation metrics

- **Pairwise accuracy** on non-tie test pairs: fraction where $\text{sign}(m)$ matches the annotator. This is the standard metric for reward models and RewardBench, and it is what matters downstream (picking the better response).
- **95% bootstrap CI** (1,000 resamples) on accuracy.
- **Paired bootstrap** on the accuracy difference between two models scored on the same pairs (e.g. full minus length-only), so model comparisons account for shared test-set noise.
- **Stratified accuracy** by length relation (chosen longer / shorter) and by |length difference| quartile, to separate content signal from the length confound.

Limitations. Accuracy ignores confidence (a 51% and a 99% prediction count the same); log-loss or calibration error would capture this and are planned for Phase 3. It treats all pairs equally although ~30% are near-ties. UltraFeedback labels are GPT-4 judgments, not human ones, so "accuracy" partly measures agreement with GPT-4's own (length-biased) preferences. Tied pairs have no correct answer and are excluded.

5. Experiments and Results

5.1 Setup

	Run 1	**Run 2**
Train	54,581 non-tie pairs (official train)	43,973 pairs (pooled, re-split)
Test	895 non-tie pairs (official test)	4,000 pairs (prompt-hash split)
Features	5 (words, lines, code, coverage, TF-IDF)	6 (log tokens, lines, bullets, Jaccard, overlap, TF-IDF)
Regularization	C = 1	C = 0.01, chosen on validation (accuracy flat across grid)
Transfer	—	HH-RLHF test, 2,000 pairs (seeded sample) and all 8,505
Seed	0	0

5.2 Main results (UltraFeedback, held out)

BT = Bradley-Terry (Section 4.1). *BT full* uses all features; *BT length-only* uses response length alone.

Model	Run 1: official test (n = 895)	Run 2: re-split test (n = 4,000)
Random	50.0%	50.0%
BT length-only	56.4% [53.2, 59.8]	56.8% [55.1, 58.3]
BT full	**60.4% [57.3, 63.7]**	**59.2% [57.7, 60.8]**
Full − length-only (paired)	+4.0 pts [+1.8, +6.2]	+2.4 pts [+0.7, +4.2]

Both runs agree: surface features reach **~60%**, length alone accounts for **~6.5 of the ~10 points above chance**, and the remaining features add a small but significant 2–4 points (both paired CIs exclude 0).

Learned weights (Run 1, standardized): `n_words` +0.264, `n_lines` +0.211, `has_code` +0.111, `tfidf_cos` +0.032, `prompt_coverage` −0.007. Length and formatting dominate; prompt relevance is nearly unused.

5.3 Cross-corpus transfer (Run 2, trained on UltraFeedback only)

Model	HH-RLHF, 2,000 seeded sample	HH-RLHF, all 8,505
BT length-only	52.5% [50.2, 54.8]	51.9% [50.9, 53.0]
BT full	51.9% [49.8, 54.3]	51.0% [49.9, 52.0]

Both models are at chance on HH-RLHF, and the extra features do not beat length. UltraFeedback's surface preferences (longer, more lines, code blocks) do not describe what HH annotators prefer.

5.4 Quantitative error analysis (Run 1, `python analyze.py`)

Confusion matrix (rows = true, cols = predicted). Errors are balanced between A and B (181 vs 173), confirming the symmetric model has no position bias.

BT full	pred A	pred B
true A	271	181
true B	173	270

By length relation:

Slice	n	BT full	Length-only
Chosen longer	500	94.2%	100.0%
Chosen shorter or equal	395	17.7%	1.3%

By |word-length difference| quartile:

|len diff| (words)	n	BT full	Length-only
0–24	229	54.6%	48.9%
24–69	220	60.5%	53.2%
69–136	222	61.7%	58.1%
136–726	224	65.2%	65.6%

The model is essentially a length detector: 94% right when the preferred answer is longer, 18% when it is shorter. Its real gain over length-only comes from pairs of **similar length** (+5.7 pts in the smallest quartile), where length cannot decide. Run 2 shows the same pattern on 4,000 pairs: chosen-shorter 36.6%, similar-length 56.8% (vs 49.8% length-only), chosen-longer 76.2%.

5.5 Qualitative error analysis

Most confident errors (both runs):

Pair	Margin	What happened
Run 1 #737	−1.69	Prompt asks for 'True'/'False'. Chosen = "True." (1 word); rejected = 727 words echoing the passage. Length wins.
Run 1 #441	+1.52	String task. Model picks a 536-word "I'm an AI language model…" walkthrough over the 148-word answer GPT-4 preferred.
Run 1 #820	+2.01	Entailment task. Model picks a 491-word generic explanation of entailment categories over the 73-word answer GPT-4 preferred.
Run 2 uf-train-61820	−1.59	Prompt asks for a headline. Chosen is one line; rejected adds a 480-word article.
Run 2 uf-train-24849	−1.91	Chosen echoes the user's AppleScript (high overlap); rejected is a verbose "I must point out…" reply.
Run 2 uf-train-49574	−1.52	Both long; rejected has 35 bullets vs 5 and an "As a respectful and helpful AI assistant…" preamble.

Recurring pattern: the model rewards length and formatting (lines, bullets, code blocks) with no notion of **correctness** or **instruction compliance**. Short correct answers to closed-form prompts (True/False, labels, headlines) are its worst failure mode. **Ambiguous cases:** many errors are near-ties (score gap ≤ 0.5) where GPT-4's choice is itself debatable; these are annotation noise rather than model failure.

Mitigations. (i) A model that reads the content — a fine-tuned language-model reward or frozen representations (Phase 3) — is needed for correctness. (ii) Features for instruction compliance (expected answer format, prompt-requested length). (iii) Reporting length-stratified accuracy so length gains are not mistaken for quality.

6. Changes Made During Phase 2

Reviewing the Run 2 code found two sampling bugs, now fixed; all Run 2 numbers above are after the fix. The HH-RLHF transfer test had sampled harmlessness pairs only (48.2% / 43.5% before, 51.9% / 52.5% after, which removed the earlier "HH prefers shorter responses" conclusion), and the saved test set depended on `--max-train`. Details, verification and before/after tables are in `REVIEW_AND_FIXES.md`.

Open decision. The two runs use different test sets (official 895-pair split vs 4,000-pair re-split). Later baselines will be scored on the official split, so Run 1 is the like-for-like reference; Run 2's larger test set and transfer results serve as robustness checks.

7. How to Run (Reproduction Procedure)

No data files are submitted. Both runs download their data from the Hugging Face Hub on first use and cache it locally; no account or token is needed (all datasets are public). Both procedures were tested from an empty cache on macOS (CPU only) and reproduce the numbers in Section 5 exactly.

7.1 Requirements

- Python 3.12 (Run 2 also runs on 3.10). One virtual environment per run; they pin different scikit-learn versions.
- Internet access on the first run: about 770 MB is downloaded into the cache (UltraFeedback, HH-RLHF, and the Qwen2.5 tokenizer used by Run 2).
- No GPU. Run 1 takes about 3 minutes and Run 2 about 2.5 minutes, including downloads.

7.2 Where the data comes from

Data	Hugging Face ID	Loaded by	Used in
UltraFeedback (binarized)	`trl-lib/ultrafeedback_binarized`	`load_dataset(...)` in each `data.py`	Runs 1 and 2
HH-RLHF (test split)	`Anthropic/hh-rlhf`	`load_hh()` in Run 2's `data.py`	Run 2 transfer
Qwen2.5 tokenizer	`Qwen/Qwen2.5-1.5B-Instruct`	`AutoTokenizer` in `features.py`	Run 2 `log_len`

Files are cached in `~/.cache/huggingface/` by default. To put the cache elsewhere, set `HF_HOME` before running:

```
export HF_HOME=/path/to/hf_cache
```

Offline machine. Run the commands below once on a machine with internet access, copy the cache folder across, and set `HF_HOME` to it on the offline machine along with `HF_HUB_OFFLINE=1` and `HF_DATASETS_OFFLINE=1`:

```
python -c "from datasets import load_dataset; load_dataset('trl-lib/ultrafeedback_binarized'); load_dataset('Anthropic/hh-rlhf', split='test')"
python -c "from transformers import AutoTokenizer; AutoTokenizer.from_pretrained('Qwen/Qwen2.5-1.5B-Instruct')"
```

7.3 Run 1: official test split

From the Run 1 folder (`data.py`, `classical.py`, `analyze.py`):

```
python3.12 -m venv .venv
.venv/bin/pip install datasets==5.1.0 pandas==3.0.6 scikit-learn==1.9.1
.venv/bin/python data.py                  # dataset statistics -> outputs/data_stats.json
(Section 3.2)
.venv/bin/python classical.py             # trains BT full + length-only, prints test
accuracy and weights
.venv/bin/python analyze.py outputs/preds_length.csv outputs/preds_classical.csv
```

`analyze.py` prints the accuracy with 95% CIs, the paired difference, the confusion matrix, the length buckets and quartiles, and the most confident errors (Sections 5.2, 5.4 and 5.5). Expected output:

```
classical test acc (non-tie, n=895): 0.6045
length    test acc (non-tie, n=895): 0.5642
classical - length: +0.0402  95% CI [+0.0179, +0.0615]
```

7.4 Run 2: re-split test set and HH-RLHF transfer

From the Run 2 folder (`bt_baseline.py`, `features.py`, `data.py`, `requirements.txt`):

```
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python bt_baseline.py --seed 0                    # Sections 5.2, 5.3, 5.5
.venv/bin/python bt_baseline.py --seed 0 --hh-test 100000   # transfer on the full HH-
RLHF test split
.venv/bin/python bt_baseline.py --predict                   # score a hand-typed pair
(demo)
```

On Windows use `.venv\Scripts\python.exe` in place of `.venv/bin/python`. Each run writes `results/bt_seed0.json` (all metrics), `results/bt_seed0.joblib` (the model) and `results/splits/uf_test_seed0.jsonl` (the frozen test set). Expected output:

```
  BT full               acc=0.592  95% CI [0.577, 0.608]  n=4000
  BT length-only        acc=0.568  95% CI [0.551, 0.583]  n=4000
  full - length-only    diff=+0.024  95% CI [+0.007, +0.042]
  BT full               acc=0.519  95% CI [0.498, 0.543]  n=2000      (HH-RLHF)
  BT length-only        acc=0.525  95% CI [0.502, 0.548]  n=2000      (HH-RLHF)
```

`--predict` loads the saved model and asks for a prompt and two responses, then prints P(A > B) for both models and each feature's contribution to the margin.

8. Next Steps (Phase 3)

- Add the modern (transformer-based) baseline, scored on the same 895 test pairs.
- Run seeds 1–2 for Run 2 so CIs cover split variation, not just test sampling.
- Build the proposed frozen-representation reranker and evaluate it against the classical baseline and the length-only control on both corpora, with paired bootstrap CIs.